

03 Design Practice

1 Introduction

These two examples are very similar to what students can expect on the midterm. They should try to solve the second exercise in 45 minutes, that's the amount of time they will have on the midterm.

On the midterm, students can choose whether to just call algorithmic patterns like `SEARCH(quiver, cond(a) -> a.type = type)`, or implement them themselves. The former is preferred. If they call them, they have to use the correct notation. If they choose to implement them themselves, they have to use enumerators (foreach loops), and have a correct implementation for each. Half-baked implementations or ones that have errors are worth 0 points.

The first assignment will also be similar to the midterm, if someone wants to practice they can solve additional ones besides the one assigned to them.

2 Ranger

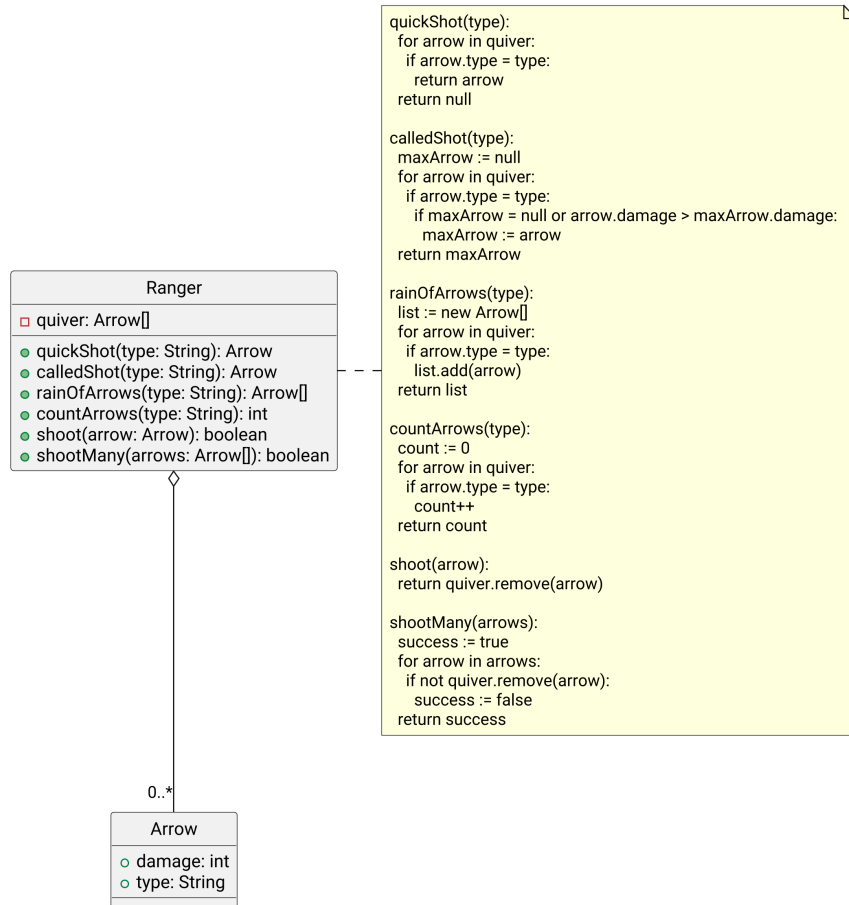
A Ranger has a quiver of arrows (list of Arrow objects). Each arrow has a damage (int) and type (String). The type could be for example broadhead arrows for unarmored foes, bodkin arrows against plate mail, or lightning arrows which channel lightning strikes upon impact. The behavior of the arrow doesn't have to be implemented.

The ranger has different combat moves which require them to look for different arrows. When they do a quick shot, they take the first arrow of a given type. When they have more time they do a called shot: they look for an arrow with the given type that does maximum damage. Rain of arrows returns all of the arrows with a given type. They would also like to know the number of arrows they have of a given type. The ranger shoots the arrows using separate methods. They have a shoot method which shoots a single arrow, and a shootMany method which shoots several arrows at once.

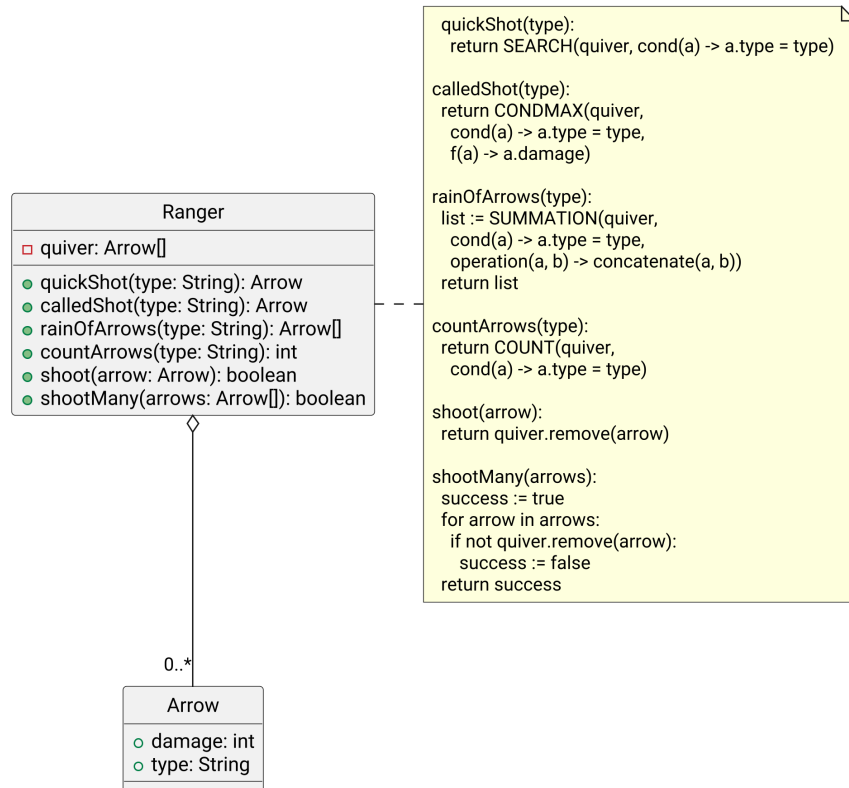
Design the Arrow and Ranger classes. You can have the fields of the Arrow class public so you don't have to do getters and setters. Do it without and with algorithmic patterns.



Solution without algorithmic patterns:



Solution with algorithmic patterns:



3 Cake Shop

A cake shop has various cakes. Each cake has a name, a color, a weight, and a cost. The customers arriving at the shop have different needs. Some of them would like to get the first cake with a specific color, and some of them would like the largest (heaviest) cake. There are also some customers who want the cheapest cake for people they are on bad terms with.

The shop owner needs a `CakeShop` class which helps them keep inventory and also find these cakes quickly. They would also like to count how many cakes of a color they have, and they would also like to get the total worth of their inventory of cakes. They can of course register new cakes into the inventory and sell the cakes. They would also like to know the total number of sold cakes.

Design the `Cake` and `CakeShop` classes. You can have the fields of the `Cake` class public so you don't have to do getters and setters. Do it without and with algorithmic patterns.

